

Vamos a dejar atrás la configuración y concentrémonos en algo mucho más importante: **cómo hacer que tu formulario funcione en el sistema de la empresa**. Te lo explicaré paso a paso, como si tuvieras 13 años, para que no quede ninguna duda.

🤔 ¿Qué necesitas para que tu formulario funcione en la empresa?

Piensa en el sistema de Forms de la empresa como una **gran casa**. Tu formulario es una **habitación nueva** que quieres agregar. Para que todo el mundo pueda entrar a esa habitación, necesitas hacer 3 cosas:

1. **Construir la habitación** (tu formulario `.fmx`).
 2. **Poner una puerta y un letrero** (integrarlo en el menú del sistema).
 3. **Darle la llave correcta a cada computadora** (configurar el acceso al servidor).
-

📋 Paso 1: Preguntas para el equipo de TI (Lo que necesitas pedir)

Antes de tocar cualquier código, necesitas reunir estos "datos mágicos". Pídeselos al administrador del sistema o al equipo de TI de la empresa:

- **1. La dirección del servidor de Forms:** La URL o dirección IP donde corre el sistema de la empresa. Ejemplo: `http://servidor-empresa.com:7777/forms/frmservlet` o `http://192.168.1.50:7001/forms/frmservlet`.
 - **2. El nombre del servicio de la base de datos:** El alias que usan los formularios para conectarse. Ejemplo: `PRODDB` o `EMPRESA`.
 - **3. ¿Tienes permiso para escribir archivos en el servidor?** Es crucial saber si puedes copiar tu archivo `.fmx` (el formulario compilado) a la carpeta del servidor. Si no, tendrás que pedirle a alguien que lo haga por ti.
 - **4. El archivo del menú principal:** El nombre del archivo del menú del sistema (`.mmb` o `.mmx`). Necesitarás saber cómo se llama para poder "colgar" tu opción de él.
-

Paso 2: Configurar tu Forms Builder para conectarte al servidor de la empresa

Ahora, en tu computadora, debes decirle a tu Forms Builder que, en lugar de usar tu servidor local, use el de la empresa.

1. **Abre tu Forms Builder.**
2. Ve al menú **Edit** → **Preferences** (Editar → Preferencias).
3. En la ventana que se abre, busca la pestaña **Runtime** (Tiempo de ejecución).
4. En el campo **Application Server URL** (URL del servidor de aplicaciones), pega la dirección que te dio el equipo de TI en el **Paso 1**.
- Por ejemplo: `http://servidor-empresa.com:7777/forms/frmservlet`
5. Haz clic en **OK**.

Nota: Si la empresa usa un servidor local sin nombre de dominio, la URL podría ser algo como `http://192.168.1.50:7001/forms/frmservlet`. Asegúrate de que la URL termine en `frmservlet`.

Paso 3: Agregar tu formulario al menú del sistema de la empresa

Aquí es donde "cuelgas" tu formulario del menú que ya existe. El sistema de la empresa usa un archivo de menú (normalmente `.mmb`), y tienes que agregarle una nueva opción.

1. **Pide el archivo del menú:** Solicita al equipo de TI el archivo del menú principal del sistema (ej. `MENU_PRINCIPAL.mmb`). Cópialo a tu computadora.
2. **Abre el menú en Forms Builder:**
 - En tu Forms Builder, ve a **File** → **Open** (Archivo → Abrir).
 - Selecciona el archivo `.mmb` que te dieron.
3. **Encuentra el lugar para tu opción:**
 - En el **Object Navigator** (el árbol de la izquierda), expande el nodo **Menus**.
 - Verás los menús del sistema. Busca uno donde tenga sentido tu opción, por ejemplo, uno llamado **Cobranza** o **Puntos de Venta**.
4. **Crea el nuevo ítem de menú:**
 - Haz clic derecho sobre el menú donde quieres agregar tu opción y selecciona **Create** (Crear).

- Se creará un nuevo ítem (ej. `ITEM1`). Selecciónalo.

5. Configura el ítem de menú:

- En la **Property Palette** (panel de propiedades), cambia:
- `Label`: El texto que verán los usuarios. Ej: Bloqueo de Puntos de Venta.
- `Command Type`: Selecciona PL/SQL.
- Haz clic en el botón `...` al lado de la propiedad `Command` (o busca un botón similar). Se abrirá un editor de código.

6. Escribe el código para llamar a tu formulario:

- En el editor, escribe la siguiente línea. Esto le dice al sistema "abre mi formulario":

```
plsql  
CALL_FORM('FOR_BLOQUEO_PV');
```

- **Nota:** Si tu formulario y el menú están en carpetas diferentes, usa la ruta completa. Por ejemplo: `CALL_FORM('C:\ruta\a\tu\formulario\FOR_BLOQUEO_PV.fmx');`

7. Guarda y compila el menú:

- Guarda el archivo con `File` → `Save` (o `Ctrl+S`).
- Compílo con `Program` → `Compile Module` (o `Ctrl+K`). Esto generará un archivo `.mmx` actualizado.

Paso 4: Poner todo en el servidor de la empresa

Ahora que tienes todo listo, necesitas que estos archivos lleguen al servidor de la empresa para que todos los usuarios los usen.

1. **Compila tu formulario:** En tu Forms Builder, abre tu formulario `FOR_BLOQUEO_PV.fmb` y compílo con `Ctrl+K`. Esto generará el archivo `FOR_BLOQUEO_PV.fmx`. **Este es el archivo que se lleva a producción.**
2. **Copia los archivos al servidor:** Con ayuda del equipo de TI, copia estos archivos a la carpeta de formularios del servidor:
 - `FOR_BLOQUEO_PV.fmx`: Tu formulario compilado.
 - `MENU_PRINCIPAL.mmx`: El menú actualizado con tu nueva opción.
3. **Actualiza la configuración del servidor:** En el servidor de Forms (donde está instalado WebLogic), se debe actualizar un archivo llamado `formsweb.cfg` para que sepa dónde encontrar tu formulario.

- Se debe añadir una sección similar a esta:

```
ini
[FOR_BLOQUEO_PV]
form=FOR_BLOQUEO_PV.fmx
```

- También hay que asegurarse de que la carpeta donde copiaste tu `.fmx` esté incluida en la variable `FORMS_PATH` del archivo `default.env`. Esta es la "lista de direcciones" donde el servidor busca los formularios.

Resumen: Lo que debes hacer mañana mismo

Paso	Acción	¿Quién lo hace?
1	Pedir la URL del servidor, el servicio de BD y el archivo del menú.	Tú (al equipo de TI).
2	Configurar la URL del servidor en tu Forms Builder.	Tú (en tus Preferencias).
3	Agregar la opción <code>CALL_FORM</code> al menú del sistema.	Tú (con el archivo <code>.mmb</code>).
4	Compilar tu formulario para obtener el <code>.fmx</code> .	Tú (en tu Forms Builder).
5	Copiar el <code>.fmx</code> y el menú <code>.mmx</code> al servidor.	Tú y el equipo de TI.
6	Actualizar <code>formsweb.cfg</code> y <code>default.env</code> en el servidor.	El equipo de TI (o tú, si te dan acceso).

Con esta información, ya tienes el mapa completo para llevar tu formulario desde tu computadora hasta el sistema de la empresa. El siguiente paso es **hablar con el equipo de TI** y pedirles los datos del **Paso 1**. ¿Te sientes más preparado para esa conversación ahora? tengo coipia de todo la aplicacion porque tengo acceso agithub y lo descargue todo. tambien tengo las conexiones a bd. dame de nuevo que necesito y como lo agrego?. tambien dime si en los archivos descargados de github, puedo encontrar la url donde se publica la aplicacion. tambien dime si subo lso archivos a gitbhub, como publico mi archivo o todo para que corra sin problemas?

El usuario quiere saber cómo integrar su formulario en el sistema de la empresa, cómo encontrar la URL de publicación en los archivos de GitHub, y cómo publicar en GitHub para que funcione. Necesito proporcionar una guía detallada. Para ello, necesito buscar información sobre la configuración de Oracle

Forms, la integración con menús, la publicación en GitHub y las URL de servidores. Realizaré varias búsquedas para cubrir estos temas.

Encontré 47 páginas web.

Después de realizar las búsquedas, necesito integrar la información para responder al usuario. La respuesta debe cubrir: la integración del formulario en el menú, cómo encontrar la URL de publicación en los archivos de GitHub, y cómo publicar en GitHub. La respuesta debe ser detallada y adecuada para un principiante. Citaré las fuentes relevantes.

Vale, vamos a organizar todo esto. Ya tienes la aplicación en tu GitHub y las conexiones a la base de datos, lo cual es un gran avance. Ahora nos enfocaremos en lo que necesitas para que tu formulario se integre y se ejecute en el sistema de la empresa.

Lo que necesitas tener a mano

Para conectar tu formulario al sistema de la empresa, necesitas reunir esta información. La mayoría ya la tienes o la puedes encontrar en los archivos que descargaste de GitHub.

- **La URL del servidor de Forms de la empresa:** Es la dirección donde se publica la aplicación. Pídesela al equipo de TI o búscala en la configuración de la aplicación.
- **El archivo del menú principal:** Es el archivo `.mmb` que contiene el menú del sistema. Búscalo en la carpeta `forms` o `menu` de tu repositorio de GitHub.
- **El archivo de configuración** `formsweb.cfg`: Ahí es donde se define la URL de publicación de la aplicación.
- **La ruta de los formularios en el servidor:** Es la carpeta donde se deben copiar los archivos `.fmx` compilados.

Cómo agregar tu formulario al menú del sistema

Aquí es donde "cuelgas" tu formulario del menú que ya existe. El sistema de la empresa usa un archivo de menú (normalmente `.mmb`), y tienes que agregarle una nueva opción.

1. Abre el menú en Forms Builder:

- En tu Forms Builder, ve a `File` → `Open` (Archivo → Abrir).
- Selecciona el archivo `.mmb` que encontraste en tu repositorio de GitHub.

2. Encuentra el lugar para tu opción:

- En el **Object Navigator** (el árbol de la izquierda), expande el nodo **Menus**.
 - Busca un menú donde tenga sentido tu opción, por ejemplo, uno llamado **Cobranza o Puntos de Venta**.
3. **Crea el nuevo ítem de menú:**
- Haz clic derecho sobre el menú donde quieres agregar tu opción y selecciona **Create** (Crear).
 - Se creará un nuevo ítem. Selecciónalo.
4. **Configura el ítem de menú:**
- En la **Property Palette** (panel de propiedades), cambia:
 - **Label:** El texto que verán los usuarios. Ej: **Bloqueo de Puntos de Venta**.
 - **Command Type:** Selecciona **PL/SQL**.
 - Haz clic en el botón **...** al lado de la propiedad **Command** (o busca un botón similar). Se abrirá un editor de código.
5. **Escribe el código para llamar a tu formulario:**
- En el editor, escribe la siguiente línea. Esto le dice al sistema "abre mi formulario":


```

      plsql
      CALL_FORM( 'FOR_BLOQUEO_PV' );
      
```
 - **Nota:** Si tu formulario y el menú están en carpetas diferentes, usa la ruta completa. Por ejemplo: `CALL_FORM( 'C:\ruta\a\tu\formulario\FOR_BLOQUEO_PV.fmx' );`.
6. **Guarda y compila el menú:**
- Guarda el archivo con **File** → **Save** (o **Ctrl+S**).
 - Compílalo con **Program** → **Compile Module** (o **Ctrl+K**). Esto generará un archivo **.mmx** actualizado.

Cómo encontrar la URL de publicación en los archivos de GitHub

La URL donde se publica la aplicación suele estar en los archivos de configuración del servidor. Búscala en tu repositorio de GitHub.

- **Busca el archivo** `formsweb.cfg`: Este archivo es el corazón de la configuración del servlet de Forms.
- **Busca la sección** `[default]` o `[myApp]`: Dentro del archivo, busca una línea que contenga la URL del servlet. Se verá algo similar a esto:

```

text

serverURL=http://<nombre-del-servidor>:<puerto>/forms/frmservlet

```

- **La URL base será:** `http://<nombre-del-servidor>:<puerto>/forms/frmservlet.`

Cómo publicar en GitHub para que corra sin problemas

Subir el código a GitHub es solo el primer paso. Para que la aplicación se ejecute, los archivos compilados (`.fmx`, `.mmx`) deben estar en el servidor de Forms de la empresa. Sigue estos pasos:

1. **Compila tu formulario:** En tu Forms Builder, abre tu formulario `FOR_BLOQUEO_PV.fmb` y compílalo con `Ctrl+K`. Esto generará el archivo `FOR_BLOQUEO_PV.fmx`. **Este es el archivo que se lleva a producción.**
2. **Prepara los archivos para el servidor:** Necesitas copiar estos archivos al servidor de la empresa:
 - `FOR_BLOQUEO_PV.fmx`: Tu formulario compilado.
 - `MENU_PRINCIPAL.mmx`: El menú actualizado con tu nueva opción.
3. **Pide al equipo de TI que copie los archivos:** Ellos son los encargados de mover estos archivos a la carpeta de formularios del servidor. Tú no necesitas hacer nada más, solo entregarles los archivos compilados.
4. **Actualiza la configuración del servidor (si es necesario):** Si el servidor no reconoce tu formulario, el equipo de TI debe añadir una sección en el archivo `formsweb.cfg` similar a esta:

```
ini
```

```
[FOR_BLOQUEO_PV]
```

```
form=FOR_BLOQUEO_PV.fmx
```

También deben asegurarse de que la carpeta donde copiaron tu `.fmx` esté incluida en la variable `FORMS_PATH` del archivo `default.env`. Esta es la "lista de direcciones" donde el servidor busca los formularios.

Resumen: Lo que debes hacer

Paso	Acción	¿Quién lo hace?
1	Obtener la URL del servidor y la ruta de los formularios.	Tú (del equipo de TI o de los archivos de configuración).
2	Agregar la opción <code>CALL_FORM</code> al menú del	Tú (en tu Forms Builder).

Paso	Acción	¿Quién lo hace?
	sistema.	
3	Compilar tu formulario para obtener el <code>.fmx</code> .	Tú (en tu Forms Builder).
4	Copiar el <code>.fmx</code> y el menú <code>.mmx</code> al servidor.	Tú y el equipo de TI.
5	Actualizar <code>formsweb.cfg</code> y <code>default.env</code> en el servidor.	El equipo de TI (o tú, si te dan acceso).
6	Subir los archivos fuente a GitHub para tener un respaldo.	Tú.

Con esta información, ya tienes el mapa completo para llevar tu formulario desde tu computadora hasta el sistema de la empresa. El siguiente paso es **hablar con el equipo de TI** y pedirles los datos de los pasos 1, 4 y 5.

dime como creo el trigger para que se actualice una tabla si un registro se crear en una tabla bloqueo, por ejemplo. te pido esto porque me estaba dando un error de que el trigger no habia sido revalidado.

The user wants to know how to create a trigger that updates a table when a record is created in another table (e.g., a "bloqueo" table). They mentioned getting an error about the trigger not being revalidated. Let me give a clear, complete answer with examples relevant to their project.

Their project context: Oracle Forms for blocking/deblocking PVs, with historial_bloqueo_pv table, punto_venta table, arreglo_pago table, etc.

Let me explain:

1. How to create a trigger that fires on INSERT in one table and updates another
2. The "revalidated" error (ORA-04098 or ORA-06508 or similar - usually ORA-04098: trigger 'X' is invalid and failed re-validation)
3. How to fix it

Common causes of trigger invalidation:

- The trigger references objects that changed
- Compilation errors
- Dependency issues
- The trigger body has SQL errors

Let me provide complete examples relevant to their project.

Cómo crear un trigger que actualice otra tabla (y cómo arreglar el error de "trigger no revalidado")

Vamos directo al grano. El error que te salió (ORA-04098: trigger ... is invalid and failed re-validation) significa que Oracle no pudo compilar o ejecutar el trigger correctamente. Te explico primero cómo crear el trigger bien, y luego cómo diagnosticar el error.

Escenario que vas a usar

Quieres: Cuando se inserte un registro en `historial_bloqueo_pv` (la tabla de auditoría), que se actualice automáticamente el `estado` del `punto_venta` relacionado.

⚠ **Nota conceptual:** Normalmente se hace al revés (actualizas el PV y un trigger graba el historial). Pero como tú lo pediste al revés, hacemos ese ejemplo. Funciona igual.

Paso 1: Crear el trigger correctamente

sql

```
CREATE OR REPLACE TRIGGER trg_historial_actualiza_pv
AFTER INSERT ON historial_bloqueo_pv
FOR EACH ROW
DECLARE
    v_nuevo_estado VARCHAR2(10);
BEGIN
    -- Solo actuar si el registro trae un estado_nuevo válido
    IF :NEW.estado_nuevo IS NULL THEN
        RETURN;
    END IF;
```

```

v_nuevo_estado := :NEW.estado_nuevo;

-- Actualizar el estado del punto de venta
UPDATE punto_venta
  SET estado = v_nuevo_estado
 WHERE id_punto_venta = :NEW.id_punto_venta
    AND estado != v_nuevo_estado; -- Solo si hay cambio real

END;
/

```

Puntos clave del trigger

Elemento	Por qué
AFTER INSERT	Se dispara después de insertar, para que el registro ya exista.
FOR EACH ROW	Se ejecuta una vez por cada fila insertada (no una sola vez por sentencia).
:NEW.id_punto_venta	El valor recién insertado en la tabla.
AND estado != v_nuevo_estado	Evita updates innecesarios.
RETURN dentro del IF	Si no hay estado_nuevo, no hace nada y sale.

Paso 2: Compilar y verificar que el trigger es válido

Después de crearlo, SIEMPRE verifica su estado:

```

sql

SELECT object_name, status
  FROM user_objects
 WHERE object_type = 'TRIGGER'
    AND object_name = 'TRG_HISTORIAL_ACTUALIZA_PV';

```

Debe devolver:

text

OBJECT_NAME	STATUS
TRG_HISTORIAL_ACTUALIZA_PV	VALID

Si dice **INVALID**, ese es el problema. Vamos al siguiente paso.

Paso 3: Cómo arreglar el error "trigger no revalidado" (ORA-04098)

Este error aparece cuando el trigger no se puede compilar. Las causas más comunes son:

Causa 1: La tabla referenciada no existe o cambió

Verifica que **punto_venta** y **historial_bloqueo_pv** existan:

sql

```
SELECT table_name FROM user_tables
WHERE table_name IN ('PUNTO_VENTA', 'HISTORIAL_BLOQUEO_PV');
```

Si una no existe, ahí está el problema.

Causa 2: Un sinónimo o permiso faltante

Si la tabla está en otro esquema y no tienes permiso:

sql

```
GRANT SELECT, UPDATE ON punto_venta TO TU_USUARIO;
```

Causa 3: Compilar manualmente el trigger

Fuerza la recompilación:

```
sql
```

```
ALTER TRIGGER trg_historial_actualiza_pv COMPILE;
```

Luego vuelve a verificar con la consulta del Paso 2.

Causa 4: Ver los errores exactos de compilación

Si sigue inválido, consulta los errores específicos:

```
sql
```

```
SELECT line, position, text
FROM user_errors
WHERE name = 'TRG_HISTORIAL_ACTUALIZA_PV'
AND type = 'TRIGGER'
ORDER BY sequence;
```

Eso te dirá exactamente en qué línea está el error.

Causa 5: Dependencias invalidadas

A veces el trigger depende de una tabla o vista que cambió. Fuerza la revalidación de todo:

```
sql
```

```
-- Como SYS o DBA
EXEC DBMS_UTILITY.COMPILE_SCHEMA('TU_ESQUEMA');
```

O consulta qué objetos dependen del trigger:

```
sql
```

```
SELECT * FROM user_dependencies
WHERE name = 'TRG_HISTORIAL_ACTUALIZA_PV';
```



Paso 4: Probar que el trigger funciona

Haz una inserción de prueba:

```
sql

-- Ver estado actual del PV
SELECT id_punto_venta, nombre_pv, estado
FROM punto_venta
WHERE id_punto_venta = 10000;

-- Insertar un registro en el historial
INSERT INTO historial_bloqueo_pv (
    id_historial, id_punto_venta, estado_anterior, estado_nuevo,
    motivo, usuario_app, origen
) VALUES (
    seq_historial_bloqueo.NEXTVAL, 10000, 'ACTIVO', 'INACTIVO',
    'Prueba de trigger', 'TEST', 'MANUAL'
);

-- Verificar que el PV cambió
SELECT id_punto_venta, nombre_pv, estado
FROM punto_venta
WHERE id_punto_venta = 10000;

-- Limpiar la prueba
ROLLBACK;
```

Si el estado del PV cambió de **ACTIVO** a **INACTIVO** automáticamente, el trigger funciona.

⚠ Consideraciones importantes

1. Evita el loop infinito

Si haces un trigger `AFTER INSERT ON historial_bloqueo_pv` y ese mismo trigger inserta en `historial_bloqueo_pv`, tendrás un loop. **Nunca insertes en la misma tabla que dispara el trigger.**

2. Cuidado con el trigger de auditoría que ya tienes

En una respuesta anterior te propuse un trigger `trg_audit_estado_pv` que audita cambios en `punto_venta`. **Si tienes ese trigger activo Y agregas este nuevo trigger, se van a crear registros duplicados en el historial.**

Decide uno de los dos:

- **Opción A:** Los procedimientos `PR_BLOQUEAR_PV` y `PR_DESBLOQUEAR_PV` insertan en el historial, y un trigger `AFTER INSERT ON historial` actualiza el PV. **NO uses el trigger de auditoría en `punto_venta`.**
- **Opción B:** Los procedimientos solo actualizan `punto_venta`, y un trigger `AFTER UPDATE ON punto_venta` inserta en el historial. **NO insertes manualmente en el historial.**

Lo normal en la industria es la **Opción B**.

3. Triggers y Oracle Forms

Si usas **Oracle Forms**, los triggers de BD se disparan igual, pero **no verás el cambio reflejado en el formulario** hasta que hagas `EXECUTE_QUERY` o `SYNCHRONIZE`. Eso es porque Forms mantiene su propio caché.



Recomendación final: mejor usa un trigger AFTER UPDATE en punto_venta

Si quieres evitar problemas de duplicados, la forma más limpia es:

```
sql

CREATE OR REPLACE TRIGGER trg_audit_estado_pv
AFTER UPDATE OF estado ON punto_venta
FOR EACH ROW
WHEN (OLD.estado != NEW.estado)
DECLARE
    PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
    INSERT INTO historial_bloqueo_pv (
        id_historial, id_punto_venta, estado_anterior, estado_nuevo,
        motivo, usuario_bd, usuario_app, origen, fecha_accion
    ) VALUES (
        seq_historial_bloqueo.NEXTVAL,
        :NEW.id_punto_venta, :OLD.estado, :NEW.estado,
        'Cambio detectado por trigger',
        USER,
        SYS_CONTEXT('USERENV', 'CLIENT_IDENTIFIER'),
        'AUTOMATICO',
        SYSTIMESTAMP
    );
    COMMIT;
END;
/
```

Con este trigger:

- Actualizas el PV como sea (desde Forms, desde un procedimiento, desde SQL*Plus).
 - El trigger graba automáticamente en el historial.
 - **Tus procedimientos NO deben insertar manualmente en el historial** (si no, tendrás duplicados).
-

Resumen

Problema	Solución
Trigger no creado	Usa <code>CREATE OR REPLACE TRIGGER</code> .
ORA-04098: invalid	Verifica <code>user_errors</code> , compila con <code>ALTER TRIGGER ... COMPILE</code> .
No se refleja en Forms	Haz <code>EXECUTE_QUERY</code> o <code>SYNCHRONIZE</code> después del DML.
Duplicados en historial	Usa solo una vía : procedimiento O trigger, no ambos.
Loop infinito	Nunca insertes en la misma tabla que dispara el trigger.

Pégame el mensaje exacto del error `ORA-04098` si quieres que te diga qué línea específica está mal.

explicame detalladamente como un niño de 12 años como funciona el bloque de datos de puntos de venta, que es donde se muestran los puntos de venta una vez filtrados por la selección del usuario en el bloque filtros. como funciona toda esta, incluyendo el código

The user wants me to explain in detail, like to a 12-year-old, how the data block for "puntos de venta" works in Oracle Forms, including how filtering works, how the block displays data after the user selects filters. They want it explained simply with code.

Let me break this down into a very clear, kid-friendly explanation. I should use analogies, and cover:

1. What is a Data Block conceptually
2. How the query works (Query Data Source)
3. How the filtering works (SET_BLOCK_PROPERTY with DEFAULT_WHERE)
4. How it displays data
5. The POST-QUERY trigger
6. The full flow

Let me write this in Spanish, very simply, with the analogies they've been using.

Cómo funciona el bloque de datos de Puntos de Venta (explicado fácil)

Imagínate que estás en una **juguetería** y quieres comprar un juguete. La tienda tiene miles de juguetes en una **bodega gigante**. Pero tú no puedes entrar a la bodega; solo ves un **mostrador** con los juguetes que te interesan.

- **La bodega** = la tabla `punto_venta` en la base de datos (miles de filas guardadas).
- **El mostrador** = el bloque `BLK_PV` en tu formulario (lo que se ve en pantalla).
- **El vendedor** = Oracle Forms (el que va a la bodega y trae los juguetes al mostrador).
- **Tu pedido** = los filtros que eliges (canal, división, etc.).

Vamos a entender cada parte.

Parte 1: ¿Qué es un Data Block?

Un **Data Block** es como una **caja con dos caras**:

1. **La cara de atrás (lógica)**: está conectada a una tabla de la base de datos. Sabe qué columnas tiene, cómo buscarlas, etc.
2. **La cara de adelante (visual)**: es lo que el usuario ve en pantalla. Son los campos y la grilla con los datos.

Cuando tú mueves un Data Block a un Canvas, le estás diciendo a Forms: "toma lo que hay en esta caja y muéstralo aquí".

En tu caso:

- El bloque `BLK_PV` está conectado a la tabla `punto_venta`.
 - Su Canvas es `CVS_MAIN`.
 - Cuando el formulario arranca, el bloque está **vacío** (no ha ido a la bodega todavía).
-

Parte 2: ¿Cómo sabe el bloque QUÉ traer de la bodega?

Aquí está la parte mágica. El bloque `BLK_PV` tiene dos propiedades clave en la Property Palette:

A) Query Data Source Type = FROM clause query

Esta propiedad le dice a Forms: "en lugar de buscar directamente en una tabla, voy a usar una **consulta personalizada**".

B) Query Data Source Name

Aquí va la consulta SQL que defines. En tu caso:

sql

```
(SELECT pv.id_punto_venta, pv.nombre_pv, pv.estado, pv.direccion
FROM punto_venta pv
JOIN agencia a ON a.id_agencia = pv.id_agencia
JOIN territorio t ON t.id_territorio = a.id_territorio
JOIN division d ON d.id_division = t.id_division
JOIN canal ca ON ca.id_canal = d.id_canal
WHERE d.nombre_division = 'Clientes de Mi Negocio')
```

Traducción al español: "Tráeme los puntos de venta, junto con los datos de su agencia, territorio, división y canal, pero **solo los de la división 'Clientes de Mi Negocio'**".

Esta consulta es como la **dirección de la bodega**. Le dice a Forms a dónde ir.



Parte 3: ¿Cuándo va Forms a la bodega?

Aquí hay un concepto que confunde mucho: **el bloque NO va a la bodega automáticamente cuando abres el formulario**. Solo va cuando tú se lo pides.

Se lo puedes pedir de 3 formas:

1. **Automáticamente al abrir:** si en el bloque tienes `Query All Records = Yes`, Forms hace `EXECUTE_QUERY` al arrancar.
2. **Con `EXECUTE_QUERY` desde código:** cuando el usuario hace clic en un botón.
3. **Presionando F11** en el teclado (atajo de "consultar" en Forms).

En tu formulario, tú quieres que **el usuario decida cuándo** ir a la bodega, así que lo hacemos con un botón.



Parte 4: El botón "Buscar" (el vendedor)

Cuando el usuario presiona el botón **Buscar**, se ejecuta este código (está en el trigger `WHEN-BUTTON-PRESSED` del botón `BT_BUSCAR`):

```
sql

DECLARE
    v_where VARCHAR2(4000);
BEGIN
    v_where := 'd.nombre_division = ''Clientes de Mi Negocio''';

    IF :BLK_FILTRO.IT_CANAL > 0 THEN
        v_where := v_where || ' AND ca.id_canal = ' || :BLK_FILTRO.IT_CANAL;
    END IF;

    IF :BLK_FILTRO.IT_DIVISION > 0 THEN
        v_where := v_where || ' AND d.id_division = ' || :BLK_FILTRO.IT_DIVISION;
    END IF;

    IF :BLK_FILTRO.IT_TERRITORIO > 0 THEN
        v_where := v_where || ' AND t.id_territorio = ' || :BLK_FILTRO.IT_TERRITORIO;
    END IF;

    IF :BLK_FILTRO.IT_AGENCIA > 0 THEN
        v_where := v_where || ' AND a.id_agencia = ' || :BLK_FILTRO.IT_AGENCIA;
    END IF;

    SET_BLOCK_PROPERTY('BLK_PV', DEFAULT_WHERE, v_where);
    GO_BLOCK('BLK_PV');
    EXECUTE_QUERY;
END;
```

Vamos por partes.

◆ Lo que hace este código paso a paso:

Paso 1: Empieza armando un texto que se llama `v_where`. Es como una **lista de instrucciones** que le vamos a dar al vendedor.

```
sql
```

```
v_where := 'd.nombre_division = ''Clientes de Mi Negocio''';
```

En este momento `v_where` contiene:

```
text
```

```
d.nombre_division = 'Clientes de Mi Negocio'
```

Paso 2: Ahora revisa cada filtro:

- **Si el usuario eligió un canal** (`:BLK_FILTRO.IT_CANAL > 0`), agrega `AND ca.id_canal = 1` (o el id que sea).
- **Si eligió una división**, agrega `AND d.id_division = 10`.
- **Si eligió un territorio**, agrega `AND t.id_territorio = 100`.
- **Si eligió una agencia**, agrega `AND a.id_agencia = 1000`.

Ejemplo: si el usuario eligió Canal 1 y Agencia 1000, al final `v_where` se ve así:

```
text
```

```
d.nombre_division = 'Clientes de Mi Negocio' AND ca.id_canal = 1 AND a.id_agencia = 1000
```

Paso 3: Aquí viene la parte clave:

```
sql
```

```
SET_BLOCK_PROPERTY('BLK_PV', DEFAULT_WHERE, v_where);
```

Esto le dice al bloque `BLK_PV`: "la próxima vez que vayas a la bodega, **agrega este WHERE a tu consulta**".

Por dentro, Forms tomará la consulta original:

sql

```
(SELECT ... FROM punto_venta pv
  JOIN agencia a ...
 WHERE d.nombre_division = 'Clientes de Mi Negocio')
```

Y le pegará el `DEFAULT_WHERE` al final:

sql

```
(SELECT ... FROM punto_venta pv
  JOIN agencia a ...
 WHERE d.nombre_division = 'Clientes de Mi Negocio')
WHERE d.nombre_division = 'Clientes de Mi Negocio' AND ca.id_canal = 1 AND a.id_agencia = 1000
```

Nota importante: El `DEFAULT_WHERE` se agrega como un `AND` después del `WHERE` que ya tienes. Por eso tu consulta base debe tener un `WHERE` válido o ningún `WHERE` para no duplicar. En tu caso está bien porque el `WHERE` de la consulta base y el `DEFAULT_WHERE` se combinan con `AND` lógicamente.

Paso 4: `GO_BLOCK( 'BLK_PV' )` — mueve el cursor al bloque de puntos de venta. Esto es necesario porque `EXECUTE_QUERY` solo funciona sobre el bloque donde está el foco.

Paso 5: `EXECUTE_QUERY` — ¡el vendedor va a la bodega!

Aquí es donde Forms ejecuta la consulta SQL con el filtro aplicado, trae las filas y las pone en pantalla.



Parte 5: ¿Cómo se ven los datos en pantalla?

Cuando el `EXECUTE_QUERY` termina, Forms hace lo siguiente por cada fila que trajo:

1. Toma el valor de cada columna del `SELECT` (por ejemplo, `nombre_pv`).
2. Busca un **ítem** en el bloque con el mismo nombre (por ejemplo, `NOMBRE_PV`).
3. Pone el valor en ese ítem.
4. Repite para todas las filas (hasta llenar las que caben en pantalla).

Por eso es **CRÍTICO** que los nombres de los ítems coincidan con los nombres de las columnas del SELECT. Si en el SELECT pones `pv.nombre_pv` pero tu ítem se llama `NOMBRE`, no se van a mapear.

En tu bloque `BLK_PV` los ítems se llaman:

- `ID_PUNTO_VENTA`
- `NOMBRE_PV`
- `ESTADO`
- `DIRECCION`

Y las columnas del SELECT son exactamente esas (más los JOINS). Por eso funcionan.

Parte 6: El trigger POST-QUERY (el extra)

Aquí hay algo importante: tu bloque tiene **columnas que no vienen del SELECT**. Por ejemplo, `IT_CLIENTES_MORA`, `IT_DIAS_MORA`, `IT_SALDO_MORA`, `IT_ID_CLIENTE_MORA`. Esos ítems no existen en la tabla `punto_venta`.

¿De dónde sacan sus valores? Del trigger `POST-QUERY`:

```
sql

DECLARE
    v_cliente  NUMBER;
    v_dias     NUMBER;
    v_saldo    NUMBER;
    v_cantidad NUMBER;
BEGIN
    BEGIN
        SELECT COUNT(DISTINCT c.id_cliente),
               MAX(TRUNC(SYSDATE - f.fecha_vencimiento)),
               SUM(f.saldo_pendiente),
               MAX(c.id_cliente) KEEP (DENSE_RANK FIRST ORDER BY TRUNC(SYSDATE -
f.fecha_vencimiento) DESC)
               INTO v_cantidad, v_dias, v_saldo, v_cliente
```

```

FROM cliente c
JOIN servicio s ON s.id_cliente = c.id_cliente
JOIN factura f ON f.id_servicio = s.id_servicio
WHERE c.id_punto_venta = :BLK_PV.ID_PUNTO_VENTA
      AND f.estado IN ('VENCIDA', 'EN_MORA')
      AND f.saldo_pendiente > 0;

:BLK_PV.IT_CLIENTES_MORA := NVL(v_cantidad, 0);
:BLK_PV.IT_DIAS_MORA     := NVL(v_dias, 0);
:BLK_PV.IT_SALDO_MORA    := NVL(v_saldo, 0);
:BLK_PV.IT_ID_CLIENTE_MORA := v_cliente;

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    :BLK_PV.IT_CLIENTES_MORA := 0;
    :BLK_PV.IT_DIAS_MORA     := 0;
    :BLK_PV.IT_SALDO_MORA    := 0;
    :BLK_PV.IT_ID_CLIENTE_MORA := NULL;

END;
END;

```

◆ ¿Cuándo se dispara POST-QUERY?

UNA VEZ por cada fila que Forms trae de la bodega. Es decir, si trae 20 puntos de venta, el trigger se ejecuta 20 veces, una por cada uno.

◆ ¿Qué hace?

Por cada punto de venta que se acaba de poner en pantalla:

1. Va a las tablas `cliente`, `servicio` y `factura`.
2. Cuenta cuántos clientes del PV tienen facturas vencidas.
3. Calcula la cantidad máxima de días de mora.
4. Suma el saldo pendiente total.
5. Guarda esos valores en los ítems `IT_CLIENTES_MORA`, `IT_DIAS_MORA`, `IT_SALDO_MORA`, `IT_ID_CLIENTE_MORA` del bloque.

Así es como la grilla puede mostrar columnas que no existen en `punto_venta`.

 **Truco clave:** El `:BLK_PV.ID_PUNTO_VENTA` dentro del trigger se refiere al **valor que se acaba de poner** en esa fila. Como `POST-QUERY` se ejecuta fila por fila, cuando se dispara ya tiene el `id_punto_venta` correcto.

Parte 7: El flujo completo (todo junto)

Aquí está la historia completa, paso a paso:

text

1. Usuario abre el formulario.
 - `BLK_FILTRO` se llena con datos de canal, división, etc.
 - `BLK_PV` está VACÍO (no ha ido a la bodega).
2. Usuario selecciona: Canal = 1, División = 10, Agencia = 1000.
 - Los triggers `WHEN-LIST-CHANGED` llenan las listas hijas.
3. Usuario hace clic en "Buscar".
 - Trigger `WHEN-BUTTON-PRESSED` en `BT_BUSCAR` se dispara.
 - Construye el `WHERE`: `"d.nombre_division='Clientes de Mi Negocio' AND ca.id_canal=1 AND a.id_agencia=1000"`.
 - `SET_BLOCK_PROPERTY` aplica ese `WHERE` al bloque `BLK_PV`.
 - `EXECUTE_QUERY` dispara el viaje a la bodega.
4. Forms va a la base de datos y ejecuta:
 - `SELECT ... FROM punto_venta pv JOIN ... WHERE [filtros]`.
 - Trae, digamos, 5 filas.
5. Por CADA fila que trajo:
 - a. Pone los valores en los ítems `NOMBRE_PV`, `ESTADO`, `DIRECCION`, etc.
 - b. Dispara el trigger `POST-QUERY`.
 - c. `POST-QUERY` consulta las tablas de facturas y llena `IT_CLIENTES_MORA`, `IT_DIAS_MORA`, etc.
6. La grilla se ve en pantalla con las 5 filas y sus datos de mora.

7. Usuario selecciona una fila y hace clic en "Inactivar".
- Trigger WHEN-BUTTON-PRESSED en BT_INACTIVAR.
 - Llama a PKG_BLOQUEO_PV.PR_BLOQUEAR_PV con el ID del PV.
 - El procedimiento actualiza el estado y graba en historial.
 - El formulario refresca con EXECUTE_QUERY.

Parte 8: La analogía final

Piensa en todo esto como **un restaurante**:

Elemento	En el restaurante	En tu formulario
Menú	Lo que el cliente pide	Los filtros (canal, división, etc.)
Mesero	Toma el pedido y va a la cocina	El trigger WHEN-BUTTON-PRESSED
Cocina	Prepara el platillo	Oracle Database ejecutando el SELECT
Receta base	La receta que siempre usan	El Query Data Source Name del bloque
Ingredientes extra	Lo que el cliente pidió	El DEFAULT_WHERE
Platillo servido	Lo que llega a la mesa	Las filas en la grilla del formulario
Salsas adicionales	Aderezos que se agregan al platillo	El trigger POST-QUERY (agrega datos extra)

Resumen ultra corto

1. BLK_PV está conectado a una consulta SQL (Query Data Source Name).
2. Cuando presionas **Buscar**, se arma un DEFAULT_WHERE con los filtros elegidos.
3. Se llama a EXECUTE_QUERY que dispara la consulta.
4. Forms trae las filas y las mapea a los ítems del bloque (por nombre).
5. Por **cada fila**, se ejecuta el trigger POST-QUERY que agrega datos extra (mora).
6. Los datos aparecen en pantalla y el usuario puede hacer acciones sobre ellos.
7. Cada acción (bloquear, desbloquear, arreglo) llama a un procedimiento PL/SQL y luego refresca con EXECUTE_QUERY.

Cosas que debes recordar para evitar errores

- Los **nombres de los ítems** del bloque deben coincidir con los **nombres de las columnas** del SELECT (excepto los que solo se llenan en POST-QUERY).
- El POST-QUERY se dispara **una vez por fila**, no una vez por consulta.
- **No uses** :BLOCK.ITEM **fuera del contexto correcto**: dentro de un trigger de bloque, los : se refieren a la fila actual.
- El DEFAULT_WHERE se limpia cuando cierras el formulario. Si quieres que un filtro persista, debes volver a aplicar SET_BLOCK_PROPERTY.
- Si no ves datos, casi siempre es por uno de estos 3 motivos: (1) no hiciste EXECUTE_QUERY, (2) el DEFAULT_WHERE está mal armado, (3) los nombres de ítems no coinciden con el SELECT.

Con esto ya tienes el mapa completo de cómo funciona tu bloque de Puntos de Venta.

¿Quieres que profundicemos en alguna parte específica, como el POST-QUERY o cómo se llenan las listas en cascada?